

Theme Transformer

Official Implementation — Complete Codebase Reference

Repository	github.com/atosystem/ThemeTransformer
Paper	Theme Transformer: Symbolic Music Generation with Theme-Conditioned Transformer
Authors	Yi-Jen Shih, Shih-Lun Wu, Frank Zalkow, Meinard Müller, Yi-Hsuan Yang
Published	IEEE Transactions on Multimedia, 2022
License	MIT
Language	Python 3.6+ PyTorch 1.7.1

Table of Contents

1. Project Overview
2. Repository Structure
3. Requirements
4. Core Architecture (mymodel.py)
5. Transformer Implementation (myTransformer.py)
6. Vocabulary System (preprocess/vocab.py)
7. Dataset & Data Loading (preprocess/music_data.py)
8. Data Preprocessing (preprocess/prepareDataPkl.py)
9. Training Script (train.py)
10. Inference Script (inference.py)
11. Argument Parser (parse_arg.py)
12. Logger (logger.py)
13. Randomness Control (randomness.py)
14. Architecture Summary

1. Project Overview

Theme Transformer is an encoder-decoder Transformer architecture designed for theme-conditioned symbolic music generation. Given a short musical theme (melody fragment), the model generates a full multi-track MIDI piece that repeatedly embeds and varies that theme throughout the composition.

Key Innovations

- **Gated Parallel Attention (GPA):** Decoder layers that run self-attention and cross-attention in parallel, then gate them with a binary theme mask. Positions inside the theme region receive cross-attention output; positions outside receive self-attention output.
- **Theme-Aligned Positional Encoding:** Instead of sequential position indices, the cross-attention query uses positional encodings aligned to the theme token positions, helping the model learn theme-position correspondence.
- **REMI Vocabulary:** Extended REMI (Revamped MIDI) token set covering three tracks (MELODY, BRIDGE, PIANO), plus Bar, Position, Tempo, Theme_Start and Theme_End tokens.
- **Hybrid Decoder Stack:** Six decoder layers using three types — CrossOnly, Parallel-GPA, and standard Self+Cross — arranged to first align with the theme then generate fluent music.

2. Repository Structure

```
ThemeTransformer/
  ■■■ mymodel.py           # Top-level model class (myLM)
  ■■■ myTransformer.py     # Custom Transformer with GPA layers
  ■■■ train.py             # Training entry-point
  ■■■ inference.py         # Inference / music generation
  ■■■ parse_arg.py         # Training hyperparameter arguments
  ■■■ logger.py            # File + console logger
  ■■■ randomness.py        # Global random seed helper
  ■■■ requirements.txt     # Python dependencies
  ■■■ tempo_dict.json      # POP909 original tempo values
  ■■■ data_pkl/
  ■   ■■■ train_seg2_512.pkl # Pre-built training set
  ■   ■■■ val_seg2_512.pkl  # Pre-built validation set
  ■■■ trained_model/
  ■   ■■■ model_ep2311.pt   # Released checkpoint (epoch 2311)
  ■■■ theme_files/         # 29 test theme MIDI files (874-909)
  ■■■ preprocess/
    ■■■ README.md          # Data annotation guide
    ■■■ vocab.py           # REMI vocabulary + MIDI ↔ token conversion
    ■■■ music_data.py      # PyTorch Dataset / DataLoader helpers
    ■■■ prepareDataPkl.py  # Converts annotated MIDIs → training PKL
```

3. Requirements (requirements.txt)

```
kiwisolver==1.3.1
matplotlib==3.3.4
miditoolkit==0.1.14
mido==1.2.10
numpy==1.19.5
Pillow==8.3.2
pyparsing==2.4.7
python-dateutil==2.8.2
scipy==1.5.4
six==1.16.0
torch==1.7.1
typing-extensions==3.10.0.2
```

Install with: `pip install -r requirements.txt` (Python 3.6.8 recommended)

4. Core Architecture — mymodel.py

Defines the top-level language model **myLM**. It wires together token embeddings, positional encoding, the custom Transformer, and an output projection layer. Also provides temperature, top-k, and nucleus sampling helpers used during inference.

Classes

PositionalEncoding	Standard sinusoidal PE. Registers 'pe' buffer (5000 × d_model). Used for encoder input.
WordEmbedding	nn.Embedding subclass. Maps token IDs → d_model vectors. padding_idx=0.
myLM	Main model. Holds transformer_model (myTransformer.Transformer), pos_encoding, token_embedding, out

myLM Constructor Parameters

Parameter	Default	Description
ntoken	—	Vocabulary size
d_model	512	Hidden dimension
dropout	0.1	Dropout rate
num_encoder_layers	6	Number of encoder layers
xorpattern	[0,1]*3	List of booleans — 1 = use GPA gate in that decoder layer

Source Code

```
"""Theme Transformer Architecture

Author: Ian Shih
Email: yjshih23@gmail.com
Date: 2021/11/03

"""
import torch.nn as nn
import torch
import math
import numpy as np
import myTransformer

class PositionalEncoding(nn.Module):
    """
    For positional encoding in transformer.

    """
    def __init__(self, d_model, pos_enc_start=0, dropout=0.1, max_len=5000):
        super(PositionalEncoding, self).__init__()
        self.dropout = nn.Dropout(p=dropout)

        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        pe = pe.unsqueeze(0).transpose(0, 1)
        self.register_buffer('pe', pe)

    def forward(self, x):
        x = x + self.pe[:x.size(0), :]
```

```

        return self.dropout(x)

class WordEmbedding(nn.Embedding):
    """
    For Token Embedding

    """
    def __init__(self, vocab_size, embed_size=512):
        super().__init__(vocab_size, embed_size, padding_idx=0)

class myLM(nn.Module):
    """
    The overall structure of model

    """
    def __init__(self, ntoken, d_model=512, dropout = 0.1, num_encoder_layers=6, xorpattern=[0,1]*3 ...
        super(myLM, self).__init__()

        # model parameter
        self.num_encoder_layers = num_encoder_layers
        self.xorpattern = xorpattern

        # model dimension
        self.d_model = d_model
        self.dropout = dropout

        # vocab size
        self.ntoken = ntoken

        # transformer model
        self.transformer_model = myTransformer.Transformer(
            d_model = self.d_model,
            nhead=8,
            dim_feedforward=self.d_model*4,
            num_encoder_layers=num_encoder_layers,
            xor_pattern=xorpattern,
            activation = "gelu"
        )

        # positional encoding used for encoder
        self.pos_encoding = PositionalEncoding(self.d_model, self.dropout)

        # token embedding
        self.token_embedding = WordEmbedding(vocab_size=self.ntoken, embed_size=self.d_model) ...

        # output layer
        self.output_layer = nn.Linear(self.d_model, self.ntoken)

    def forward(self, src, tgt, tgt_label, tgt_mask=None, src_key_padding_mask=None, tgt_key_padding_ ...
        """forward pass of the model

        Args:
            src (tensor): src sequence
            tgt (tensor): tgt sequence
            tgt_label (tensor): information for theme-aligned positional encoding
            tgt_mask (tensor, optional): mask for cross attending encoder's output. Defaults to ...
            src_key_padding_mask (tensor, optional): src key padding mask. Defaults to None.
            tgt_key_padding_mask (tensor, optional): tgt key padding mask. Defaults to None.
            memory_mask (tensor, optional): for causal attention mask, Leave it None for bidire ...

        Returns:
            tensor: output logits
        """

        src = self.token_embedding(src)
        src = self.pos_encoding(src)
        tgt = self.token_embedding(tgt)

```

```

        out = self.transformer_model(src, tgt,
                                     tgt_mask=tgt_mask,
                                     src_key_padding_mask=src_key_padding_mask,
                                     tgt_key_padding_mask=tgt_key_padding_mask,
                                     memory_key_padding_mask=src_key_padding_mask,
                                     memory_mask=memory_mask,
                                     tgt_label=tgt_label)

    out = self.output_layer(out)

    return out

#####
# search strategy: temperature (re-shape)
#####
def temperature(self, logits, temperature):
    probs = np.exp(logits / temperature) / np.sum(np.exp(logits / temperature))
    return probs

#####
# search strategy: topk (truncate)
#####
def topk(self, probs, k):
    sorted_index = np.argsort(probs)[::-1]
    candi_index = sorted_index[:k]
    candi_probs = [probs[i] for i in candi_index]
    candi_probs /= sum(candi_probs)
    word = np.random.choice(candi_index, size=1, p=candi_probs)[0]
    return word

#####
# search strategy: nucleus (truncate)
#####
def nucleus(self, probs, p):
    probs /= sum(probs)
    sorted_probs = np.sort(probs)[::-1]
    sorted_index = np.argsort(probs)[::-1]
    cusum_sorted_probs = np.cumsum(sorted_probs)
    after_threshold = cusum_sorted_probs > p
    if sum(after_threshold) > 0:
        last_index = np.where(after_threshold)[0][-1]
        candi_index = sorted_index[:last_index]
    else:
        candi_index = sorted_index[:3] # just assign a value
    candi_probs = [probs[i] for i in candi_index]
    candi_probs /= sum(candi_probs)
    word = np.random.choice(candi_index, size=1, p=candi_probs)[0]
    return word

```

5. Transformer Implementation — myTransformer.py

Revised from PyTorch's stock Transformer. Introduces theme-aware decoding via three specialised decoder layer types and a hybrid decoder stack.

Classes Overview

Class	Role
PositionalEncoding	PE with get_segment_pos_enc() — returns PE vectors indexed by theme label positions.
Transformer	Top-level wrapper. Builds encoder (TransformerEncoder) + hybrid decoder (TransformerDecoder_Hybrid).
TransformerEncoder	Stack of N TransformerEncoderLayer. Standard bidirectional self-attention.
TransformerDecoder	Vanilla stack (not used by default — kept for reference).
TransformerDecoder_Hybrid	Hybrid stack that selects layer type per position using xor_pattern.
TransformerEncoderLayer	Standard: self-attn → Add&Norm → FFN → Add&Norm.
TransformerDecoderLayer	Standard: self-attn → cross-attn → FFN (3 sub-layers).
TransformerDecoderLayer_CrossOnly	Cross-attention only (no self-attn). Applies theme gate on cross output.
TransformerDecoderLayer_parallel	Parallel self + cross attention with XOR gate (GPA layer).

GPA (Gated Parallel Attention) Logic

In **TransformerDecoderLayer_parallel**, self-attention and cross-attention are computed in parallel on the same input. A binary gate `gate_msk` is derived from `tgt_label` (1 = inside theme region). Cross-attention output is multiplied by `gate_msk`; when `mutual_exclusive=True`, self-attention output is multiplied by `~gate_msk`. The two outputs are averaged and added to the residual connection.

Source Code

```
"""Theme Transformer

Revised from pytorch transformer implementation

Author: Ian Shih
Email: yjshih23@gmail.com
Date: 2021/11/03

"""
import torch
import copy
import math
from torch.nn import functional as F
from torch.nn.modules.module import Module
from torch.nn.modules.activation import MultiheadAttention
from torch.nn.modules.container import ModuleList
from torch.nn.init import xavier_uniform_
from torch.nn.modules.dropout import Dropout
from torch.nn.modules.linear import Linear
from torch.nn.modules.normalization import LayerNorm

class PositionalEncoding(Module):
    """
    For positional encoding in transformer.

    """
    def __init__(self, d_model, pos_enc_start=0, dropout=0.1, max_len=5000):
```



```

super(PositionalEncoding, self).__init__()
self.dropout = torch.nn.Dropout(p=dropout)
self.d_model = d_model

pe = torch.zeros(max_len, d_model)
position = torch.arange(pos_enc_start, max_len, dtype=torch.float).unsqueeze(1)
div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model ...
pe[:, 0::2] = torch.sin(position * div_term)
pe[:, 1::2] = torch.cos(position * div_term)
pe = pe.unsqueeze(0).transpose(0, 1)
self.pepe = pe
self.register_buffer('pe', pe)

def get_segment_pos_enc(self, segment_label):
    """get positional encoding according to the postional given

    Args:
        segment_label (tensor): positional numbers [seq_len, bsz]

    Returns:
        tensor: [seq_len, bsz, hid_dim]
    """
    # segment_label shape [seq_len, bsz]

    seq_len = segment_label.shape[0]
    bsz = segment_label.shape[1]
    segment_label = copy.deepcopy(segment_label)

    pos_enc = self.pe.squeeze()[segment_label]
    del segment_label
    return pos_enc

def forward(self, x, pos_enc=None):
    if not pos_enc == None:
        x = x + pos_enc
    else:
        x = x + self.pe[:x.size(0), :]

    return self.dropout(x)

class Transformer(Module):
    """
    Transformer architecture
    """

    def __init__(self, d_model=512, nhead=8, num_encoder_layers=6,
                 xor_pattern=[0,1]*3, dim_feedforward=2048, dropout=0.1,
                 activation="relu", custom_encoder=None, custom_decoder=None):
        super(Transformer, self).__init__()

        self.d_model = d_model
        self.nhead = nhead

        self.pos_enc = PositionalEncoding(self.d_model, dropout=dropout)
        if custom_encoder is not None:
            self.encoder = custom_encoder
        else:
            encoder_layer = TransformerEncoderLayer(d_model, nhead, dim_feedforward, dropout, a ...
            encoder_norm = LayerNorm(d_model)
            self.encoder = TransformerEncoder(encoder_layer, num_encoder_layers, encoder_norm)

        if custom_decoder is not None:
            self.decoder = custom_decoder
        else:
            decoder_layer_0 = TransformerDecoderLayer_CrossOnly(d_model, nhead, dim_feedforward ...

```

```

        decoder_layer_1 = TransformerDecoderLayer_parallel(d_model, nhead, dim_feedforward, ...
        decoder_layer_2 = TransformerDecoderLayer(d_model, nhead, dim_feedforward, dropout, ...

        decoder_norm = LayerNorm(d_model)

        decoder_patten = [1] * len(xor_pattern)

        self.decoder = TransformerDecoder_Hybrid([decoder_layer_0, decoder_layer_1, decoder_l ...

    self._reset_parameters()

def forward(self, src, tgt, src_mask=None, tgt_mask=None,
            memory_mask=None, src_key_padding_mask=None,
            tgt_key_padding_mask=None, memory_key_padding_mask=None, tgt_label=None):

    if src.size(1) != tgt.size(1):
        raise RuntimeError("the batch number of src and tgt must be equal")

    if src.size(2) != self.d_model or tgt.size(2) != self.d_model:
        raise RuntimeError("the feature number of src and tgt must be equal to d_model")

    att_cross_pos_enc = self.pos_enc.get_segment_pos_enc(tgt_label)
    # increasing theme mask
    # att_cross_pos_enc = (tgt_label)
    memory = self.encoder(src, mask=src_mask, src_key_padding_mask=src_key_padding_mask)
    output = self.decoder(tgt, memory, tgt_mask=tgt_mask, memory_mask=memory_mask,
                          tgt_key_padding_mask=tgt_key_padding_mask,
                          memory_key_padding_mask=memory_key_padding_mask,
                          tgt_label=tgt_label, att_cross_pos_enc=att_cross_pos_enc)

    return output

def generate_square_subsequent_mask(self, sz):
    r"""Generate a square mask for the sequence. The masked positions are filled with float ...
        Unmasked positions are filled with float(0.0).
    """
    mask = (torch.triu(torch.ones(sz, sz)) == 1).transpose(0, 1)
    mask = mask.float().masked_fill(mask == 0, float('-inf')).masked_fill(mask == 1, float( ...
    return mask

def _reset_parameters(self):
    r"""Initiate parameters in the transformer model."""

    for p in self.parameters():
        if p.dim() > 1:
            # torch.nn.init.kaiming_uniform_(p)
            # torch.nn.init.normal_(p, mean=0, std=0.01)
            xavier_uniform_(p)

class TransformerEncoder(Module):
    r"""TransformerEncoder is a stack of N encoder layers

    Args:
        encoder_layer: an instance of the TransformerEncoderLayer() class (required).
        num_layers: the number of sub-encoder-layers in the encoder (required).
        norm: the layer normalization component (optional).

    Examples::
        >>> encoder_layer = nn.TransformerEncoderLayer(d_model=512, nhead=8)
        >>> transformer_encoder = nn.TransformerEncoder(encoder_layer, num_layers=6)
        >>> src = torch.rand(10, 32, 512)
        >>> out = transformer_encoder(src)

```

```

"""

def __init__(self, encoder_layer, num_layers, norm=None):
    super(TransformerEncoder, self).__init__()
    self.layers = _get_clones(encoder_layer, num_layers)
    self.num_layers = num_layers
    self.norm = norm
    print("Encoder layers #{0}".format(self.num_layers))

def forward(self, src, mask=None, src_key_padding_mask=None):
    r"""Pass the input through the encoder layers in turn.

    Args:
        src: the sequence to the encoder (required).
        mask: the mask for the src sequence (optional).
        src_key_padding_mask: the mask for the src keys per batch (optional).

    Shape:
        see the docs in Transformer class.
    """
    output = src

    for i in range(self.num_layers):
        output = self.layers[i](output, src_mask=mask,
                                src_key_padding_mask=src_key_padding_mask)

    if self.norm:
        output = self.norm(output)

    return output


class TransformerDecoder(Module):
    r"""TransformerDecoder is a stack of N decoder layers

    Args:
        decoder_layer: an instance of the TransformerDecoderLayer() class (required).
        num_layers: the number of sub-decoder-layers in the decoder (required).
        norm: the layer normalization component (optional).

    Examples::
        >>> decoder_layer = nn.TransformerDecoderLayer(d_model=512, nhead=8)
        >>> transformer_decoder = nn.TransformerDecoder(decoder_layer, num_layers=6)
        >>> memory = torch.rand(10, 32, 512)
        >>> tgt = torch.rand(20, 32, 512)
        >>> out = transformer_decoder(tgt, memory)
    """

    def __init__(self, decoder_layer, num_layers, norm=None):
        super(TransformerDecoder, self).__init__()
        self.layers = _get_clones(decoder_layer, num_layers)
        self.num_layers = num_layers
        self.norm = norm

    def forward(self, tgt, memory, tgt_mask=None,
                memory_mask=None, tgt_key_padding_mask=None,
                memory_key_padding_mask=None):
        r"""Pass the inputs (and mask) through the decoder layer in turn.

    Args:
        tgt: the sequence to the decoder (required).
        memory: the sequence from the last layer of the encoder (required).
        tgt_mask: the mask for the tgt sequence (optional).
        memory_mask: the mask for the memory sequence (optional).
        tgt_key_padding_mask: the mask for the tgt keys per batch (optional).
        memory_key_padding_mask: the mask for the memory keys per batch (optional).

    Shape:

```

```

        see the docs in Transformer class.
"""
output = tgt

for i in range(self.num_layers):
    output = self.layers[i](output, memory, tgt_mask=tgt_mask,
                             memory_mask=memory_mask,
                             tgt_key_padding_mask=tgt_key_padding_mask,
                             memory_key_padding_mask=memory_key_padding_mask)

    if self.norm:
        output = self.norm(output)

return output

class TransformerDecoder_Hybrid(Module):
    r"""TransformerDecoder is a stack of N decoder layers

    Args:
        decoder_layer: an instance of the TransformerDecoderLayer() class (required).
        num_layers: the number of sub-decoder-layers in the decoder (required).
        norm: the layer normalization component (optional).

    Examples::
        >>> decoder_layer = nn.TransformerDecoderLayer(d_model=512, nhead=8)
        >>> transformer_decoder = nn.TransformerDecoder(decoder_layer, num_layers=6)
        >>> memory = torch.rand(10, 32, 512)
        >>> tgt = torch.rand(20, 32, 512)
        >>> out = transformer_decoder(tgt, memory)
"""

    def __init__(self, decoder_layer, decoder_layer_pattern, xor_pattern, norm=None):
        super(TransformerDecoder_Hybrid, self).__init__()

        self.layers = ModuleList([ copy.deepcopy(decoder_layer[x]) for x in decoder_layer_patte ...
        self.xor_pattern = xor_pattern
        self.num_layers = len(decoder_layer_pattern)
        self.norm = norm
        print("Decoder layers #{0}".format(self.num_layers))

    def forward(self, tgt, memory, tgt_mask=None,
                memory_mask=None, tgt_key_padding_mask=None,
                memory_key_padding_mask=None, tgt_label=None, att_cross_pos_enc=None):
        r"""Pass the inputs (and mask) through the decoder layer in turn.

        Args:
            tgt: the sequence to the decoder (required).
            memory: the sequence from the last layer of the encoder (required).
            tgt_mask: the mask for the tgt sequence (optional).
            memory_mask: the mask for the memory sequence (optional).
            tgt_key_padding_mask: the mask for the tgt keys per batch (optional).
            memory_key_padding_mask: the mask for the memory keys per batch (optional).

        Shape:
            see the docs in Transformer class.
"""
        output = tgt

        for i in range(self.num_layers):
            if self.xor_pattern[i]:
                # require xor gate
                output = self.layers[i](output, memory, tgt_mask=tgt_mask,
                                         memory_mask=memory_mask,
                                         tgt_key_padding_mask=tgt_key_padding_mask,
                                         memory_key_padding_mask=memory_key_padding_mask,
                                         tgt_label=tgt_label, mutual_exclusive=True, att_cross_pos ...
            else:

```

```

        # no require xor gate
        output = self.layers[i](output, memory, tgt_mask=tgt_mask,
                                memory_mask=memory_mask,
                                tgt_key_padding_mask=tgt_key_padding_mask,
                                memory_key_padding_mask=memory_key_padding_mask,
                                tgt_label=tgt_label, att_cross_pos_enc=att_cross_pos_enc ...)

    if self.norm:
        output = self.norm(output)

    return output

class TransformerEncoderLayer(Module):
    r"""TransformerEncoderLayer is made up of self-attn and feedforward network.
    This standard encoder layer is based on the paper "Attention Is All You Need".
    Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez,
    Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In Advances in
    Neural Information Processing Systems, pages 6000-6010. Users may modify or implement
    in a different way during application.

    Args:
        d_model: the number of expected features in the input (required).
        nhead: the number of heads in the multiheadattention models (required).
        dim_feedforward: the dimension of the feedforward network model (default=2048).
        dropout: the dropout value (default=0.1).
        activation: the activation function of intermediate layer, relu or gelu (default=relu).

    Examples::
        >>> encoder_layer = nn.TransformerEncoderLayer(d_model=512, nhead=8)
        >>> src = torch.rand(10, 32, 512)
        >>> out = encoder_layer(src)
    """

    def __init__(self, d_model, nhead, dim_feedforward=2048, dropout=0.1, activation="relu"):
        super(TransformerEncoderLayer, self).__init__()
        self.self_attn = MultiheadAttention(d_model, nhead, dropout=dropout)
        # Implementation of Feedforward model
        self.linear1 = Linear(d_model, dim_feedforward)
        self.dropout = Dropout(dropout)
        self.linear2 = Linear(dim_feedforward, d_model)

        self.norm1 = LayerNorm(d_model)
        self.norm2 = LayerNorm(d_model)
        self.dropout1 = Dropout(dropout)
        self.dropout2 = Dropout(dropout)

        self.activation = _get_activation_fn(activation)

    def forward(self, src, src_mask=None, src_key_padding_mask=None):
        r"""Pass the input through the encoder layer.

        Args:
            src: the sequence to the encoder layer (required).
            src_mask: the mask for the src sequence (optional).
            src_key_padding_mask: the mask for the src keys per batch (optional).

        Shape:
            see the docs in Transformer class.
        """
        src2 = self.self_attn(src, src, src, attn_mask=src_mask,
                              key_padding_mask=src_key_padding_mask)[0]
        src = src + self.dropout1(src2)
        src = self.norm1(src)
        if hasattr(self, "activation"):
            src2 = self.linear2(self.dropout(self.activation(self.linear1(src))))
        else: # for backward compatibility
            src2 = self.linear2(self.dropout(F.relu(self.linear1(src))))
        src = src + self.dropout2(src2)

```

```

src = self.norm2(src)
return src

```

```

class TransformerDecoderLayer(Module):
    r"""TransformerDecoderLayer is made up of self-attn, multi-head-attn and feedforward network ...
    This standard decoder layer is based on the paper "Attention Is All You Need".
    Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez,
    Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In Advances in
    Neural Information Processing Systems, pages 6000-6010. Users may modify or implement
    in a different way during application.

```

```

Args:
    d_model: the number of expected features in the input (required).
    nhead: the number of heads in the multiheadattention models (required).
    dim_feedforward: the dimension of the feedforward network model (default=2048).
    dropout: the dropout value (default=0.1).
    activation: the activation function of intermediate layer, relu or gelu (default=relu).

```

```

Examples::

```

```

>>> decoder_layer = nn.TransformerDecoderLayer(d_model=512, nhead=8)
>>> memory = torch.rand(10, 32, 512)
>>> tgt = torch.rand(20, 32, 512)
>>> out = decoder_layer(tgt, memory)
"""

```

```

def __init__(self, d_model, nhead, dim_feedforward=2048, dropout=0.1, activation="relu"):
    super(TransformerDecoderLayer, self).__init__()
    self.self_attn = MultiheadAttention(d_model, nhead, dropout=dropout)
    self.multihead_attn = MultiheadAttention(d_model, nhead, dropout=dropout)
    # Implementation of Feedforward model
    self.linear1 = Linear(d_model, dim_feedforward)
    self.dropout = Dropout(dropout)
    self.linear2 = Linear(dim_feedforward, d_model)

    self.norm1 = LayerNorm(d_model)
    self.norm2 = LayerNorm(d_model)
    self.norm3 = LayerNorm(d_model)
    self.dropout1 = Dropout(dropout)
    self.dropout2 = Dropout(dropout)
    self.dropout3 = Dropout(dropout)

    self.activation = _get_activation_fn(activation)

```

```

def forward(self, tgt, memory, tgt_mask=None, memory_mask=None,
            tgt_key_padding_mask=None, memory_key_padding_mask=None):
    r"""Pass the inputs (and mask) through the decoder layer.

```

```

Args:
    tgt: the sequence to the decoder layer (required).
    memory: the sequence from the last layer of the encoder (required).
    tgt_mask: the mask for the tgt sequence (optional).
    memory_mask: the mask for the memory sequence (optional).
    tgt_key_padding_mask: the mask for the tgt keys per batch (optional).
    memory_key_padding_mask: the mask for the memory keys per batch (optional).

```

```

Shape:
    see the docs in Transformer class.
"""

```

```

tgt2 = self.self_attn(tgt, tgt, tgt, attn_mask=tgt_mask,
                    key_padding_mask=tgt_key_padding_mask)[0]
tgt = tgt + self.dropout1(tgt2)
tgt = self.norm1(tgt)
tgt2 = self.multihead_attn(tgt, memory, memory, attn_mask=memory_mask,
                    key_padding_mask=memory_key_padding_mask)[0]
tgt = tgt + self.dropout2(tgt2)
tgt = self.norm2(tgt)
if hasattr(self, "activation"):
    tgt2 = self.linear2(self.dropout(self.activation(self.linear1(tgt))))

```

```

        else: # for backward compatibility
            tgt2 = self.linear2(self.dropout(F.relu(self.linear1(tgt))))
            tgt = tgt + self.dropout3(tgt2)
            tgt = self.norm3(tgt)
            return tgt

class TransformerDecoderLayer_CrossOnly(Module):
    """Transformer decoder layer with only cross attention

    """

    def __init__(self, d_model, nhead, dim_feedforward=2048, dropout=0.1, activation="relu"):
        super(TransformerDecoderLayer_CrossOnly, self).__init__()
        # self.self_attn = MultiheadAttention(d_model, nhead, dropout=dropout)
        self.multihead_attn = MultiheadAttention(d_model, nhead, dropout=dropout)
        # Implementation of Feedforward model
        self.linear1 = Linear(d_model, dim_feedforward)
        self.dropout = Dropout(dropout)
        self.linear2 = Linear(dim_feedforward, d_model)

        # self.norm1 = LayerNorm(d_model)
        self.norm2 = LayerNorm(d_model)
        self.norm3 = LayerNorm(d_model)
        # self.dropout1 = Dropout(dropout)
        self.dropout2 = Dropout(dropout)
        self.dropout3 = Dropout(dropout)

        # for cross attention visualization
        self.cross_att_matrix = []

        self.pos_encoding = PositionalEncoding(d_model, dropout=dropout)

        self.activation = _get_activation_fn(activation)

    def forward(self, tgt, memory, tgt_mask=None, memory_mask=None,
                tgt_key_padding_mask=None, memory_key_padding_mask=None, tgt_label=None, att_cros ...
    r"""Pass the inputs (and mask) through the decoder layer.

    Args:
        tgt: the sequence to the decoder layer (required).
        memory: the sequence from the last layer of the encoder (required).
        tgt_mask: the mask for the tgt sequence (optional).
        memory_mask: the mask for the memory sequence (optional).
        tgt_key_padding_mask: the mask for the tgt keys per batch (optional).
        memory_key_padding_mask: the mask for the memory keys per batch (optional).

    Shape:
        see the docs in Transformer class.
    """
    # tgt2 = self.self_attn(tgt, tgt, tgt, attn_mask=tgt_mask,
    #                       key_padding_mask=tgt_key_padding_mask)[0]
    # tgt = tgt + self.dropout1(tgt2)
    # tgt = self.norm1(tgt)
    tgt2 = self.pos_encoding(tgt, att_cross_pos_enc)
    tgt2, _, self.cross_att_matrix = self.multihead_attn(tgt2, memory, memory, attn_mask=me ...
                    key_padding_mask=memory_key_padding_mask)

    # gating on cross output
    if not tgt_label == None:
        tgt2 = (tgt_label.unsqueeze(2).expand(-1, -1, tgt2.shape[2])) * tgt2

    tgt = tgt + self.dropout2(tgt2)
    tgt = self.norm2(tgt)
    if hasattr(self, "activation"):
        tgt2 = self.linear2(self.dropout(self.activation(self.linear1(tgt))))
    else: # for backward compatibility
        tgt2 = self.linear2(self.dropout(F.relu(self.linear1(tgt))))
    tgt = tgt + self.dropout3(tgt2)

```

```

        tgt = self.norm3(tgt)
        return tgt

class TransformerDecoderLayer_parallel(Module):
    """Transformer with parallel cross- and self- attention together with xor gate

    """

    def __init__(self, d_model, nhead, dim_feedforward=2048, dropout=0.1, activation="relu"):
        super(TransformerDecoderLayer_parallel, self).__init__()
        self.self_attn = MultiheadAttention(d_model, nhead, dropout=dropout)
        self.multihead_attn = MultiheadAttention(d_model, nhead, dropout=dropout)
        # Implementation of Feedforward model
        self.linear1 = Linear(d_model, dim_feedforward)
        self.dropout = Dropout(dropout)
        self.linear2 = Linear(dim_feedforward, d_model)

        self.norm1 = LayerNorm(d_model)
        # self.norm2 = LayerNorm(d_model)
        self.norm3 = LayerNorm(d_model)
        self.dropout1 = Dropout(dropout)
        self.dropout2 = Dropout(dropout)
        self.dropout3 = Dropout(dropout)

        # for cross attention visualization
        self.cross_att_matrix = []

        self.pos_encoding = PositionalEncoding(d_model, dropout=dropout)

        self.activation = _get_activation_fn(activation)

    def forward(self, tgt, memory, tgt_mask=None, memory_mask=None,
                tgt_key_padding_mask=None, memory_key_padding_mask=None, tgt_label=None, mutual_e ...
    r"""Pass the inputs (and mask) through the decoder layer.

    Args:
        tgt: the sequence to the decoder layer (required).
        memory: the sequence from the last layer of the encoder (required).
        tgt_mask: the mask for the tgt sequence (optional).
        memory_mask: the mask for the memory sequence (optional).
        tgt_key_padding_mask: the mask for the tgt keys per batch (optional).
        memory_key_padding_mask: the mask for the memory keys per batch (optional).

    Shape:
        see the docs in Transformer class.
    """

    tgt2_self = self.pos_encoding(tgt)
    tgt2_self = self.self_attn(tgt2_self, tgt2_self, tgt2_self, attn_mask=tgt_mask,
                              key_padding_mask=tgt_key_padding_mask)[0]
    tgt2_self = self.dropout1(tgt2_self)

    tgt2_cross = self.pos_encoding(tgt, att_cross_pos_enc)

    tgt2_cross, _ = self.multihead_attn(tgt2_cross, memory, memory, attn_mask=memory_mask,
                                       key_padding_mask=memory_key_padding_mask)

    tgt2_cross = self.dropout2(tgt2_cross)

    gate_msk = ((tgt_label > 0).unsqueeze(2).expand(-1, -1, tgt2_cross.shape[2]))
    assert (gate_msk.dtype == torch.bool)

    gate_msk = (tgt_label.unsqueeze(2).expand(-1, -1, tgt2_cross.shape[2])).bool()

    if not tgt_label == None:

```



```

        tgt2_cross = gate_msk * tgt2_cross
        if mutual_exclusive:
            tgt2_self = ~gate_msk * tgt2_self

    tgt3 = (tgt2_self + tgt2_cross) / 2.0

    tgt3 = tgt + tgt3

    tgt = self.norm1(tgt3)

    if hasattr(self, "activation"):
        tgt2 = self.linear2(self.dropout(self.activation(self.linear1(tgt))))
    else: # for backward compatibility
        tgt2 = self.linear2(self.dropout(F.relu(self.linear1(tgt))))
    tgt = tgt + self.dropout3(tgt2)
    tgt = self.norm3(tgt)
    return tgt

def _get_clones(module, N):
    return ModuleList([copy.deepcopy(module) for i in range(N)])

def _get_activation_fn(activation):
    if activation == "relu":
        return F.relu
    elif activation == "gelu":
        return F.gelu
    else:
        raise RuntimeError("activation should be relu/gelu, not %s." % activation)

```

6. Vocabulary System — preprocess/vocab.py

The **Vocab** class defines the REMI-style token vocabulary and handles bidirectional conversion between MIDI files and token ID sequences.

Token Types

Token Group	Count	Description
Note-On-{TRACK}	$127 \times 3 = 381$	Pitch 1–127 for MELODY, BRIDGE, PIANO
Note-Duration-{TRACK}	$64 \times 3 = 192$	Duration 1–64 sub-beats per track
Note-Velocity-{TRACK}	$126 \times 3 = 378$	Velocity 1–126 per track
Tempo	60	BPM 17–197 step 3
Position	16	Sub-beat position 0–15 within a bar
Bar	1	Bar boundary marker
Phrase_Start / Phrase_End	2	Phrase region markers (unused in final paper)
Theme_Start / Theme_End	2	Theme region delimiters
padding (id=0)	1	Padding token
Total	~1,033	

Key Methods

Method	Description
build()	Constructs token2id and id2token dicts.
midi2REMI(path, ...)	Parses MIDI → sorted note/tempo events → REMI token list. Supports quantisation, track filtering, theme annotation.
preprocessREMI(seq, ...)	Slices long sequences into training windows. Returns src (theme), tgt_segments list, and binary theme mask.
REMIID2midi(ids, path)	Converts token ID list back to MIDI with MELODY + PIANO + Theme info tracks.
getPitch(token)	Extracts pitch integer from a Note-On token; returns -1 for non-pitch tokens.

Source Code

```
"""Vocabulary for theme-based transformer

Author: Ian Shih
Email: yjshih23@gmail.com
Date: 2021/11/03

"""
import pickle
import numpy as np
import miditoolkit
import os
import math
from miditoolkit.midi import parser as mid_parser
from miditoolkit.midi import containers as ct

class Vocab(object):
    def __init__(self):
        """initialize some vocabulary settings
        """
        # split each beat into 4 subbeats
```

```

self.q_beat = 4

# dictionary for matching token ID to name and the other way around.
self.token2id = {}
self.id2token = {}

# midi pitch number : 1 ~ 127 (highest pitch)
self._pitch_bins = np.arange(start=1, stop=128)

# duration tokens 1~64 of self.q_beat
self._duration_bins = np.arange(start=1, stop=self.q_beat*16+1)

# velocity tokens 1~127 (corressponding to midi format)
self._velocity_bins = np.arange(start=1, stop=127)

# tempo tokens 17~197 (determined from our dataset)
self._tempo_bins = np.arange(start=17, stop=197, step=3)

# position(subbeat) tokens 0~15, indicate the relative position with in a bar
self._position_bins = np.arange(start=0, stop=16)

self.n_tokens = 0

self.token_type_base = {}

self.tracks = ["MELODY", "BRIDGE", "PIANO"]

self.build()

# vocab
# Note-On (129) : 0 (padding) , 1 ~ 127(highest pitch) , 128 (rest)
# Note-Duration : 1 ~ 16 beat * 3
# min resolution 1/12 notes

def build(self):
    """build our vocab"""
    self.token2id = {}
    self.id2token = {}

    self.n_tokens = 0

    self.token2id['padding'] = 0
    self.n_tokens += 1

    # Note related tokens=====
    # Create Note-On tokens for each track
    for track in self.tracks:
        # Note-On
        self.token_type_base['Note-On-{}'.format(track)] = 1
        for i in self._pitch_bins:
            self.token2id['Note-On-{}_{}'.format(track, i)] = self.n_tokens
            self.n_tokens += 1

    # Create Note-Duration tokens for each track
    for track in self.tracks:
        # Note-Duration
        self.token_type_base['Note-Duration-{}'.format(track)] = self.n_tokens
        for note_dur in self._duration_bins:
            self.token2id['Note-Duration-{}_{}'.format(track, note_dur)] = self.n_tokens
            self.n_tokens += 1

    # Create Note-Velocity tokens for each track
    for track in self.tracks:
        # Note-Velocity
        self.token_type_base['Note-Velocity-{}'.format(track)] = self.n_tokens
        for vel in self._velocity_bins:
            self.token2id['Note-Velocity-{}_{}'.format(track, vel)] = self.n_tokens

```

```

        self.n_tokens += 1

# Metric related tokens===== ...
# Tempo
self.token_type_base['Tempo'] = self.n_tokens
for tmp in self._tempo_bins:
    self.token2id[ 'Tempo_{}'.format(tmp) ] = self.n_tokens
    self.n_tokens += 1

# Positions
self.token_type_base['Position'] = self.n_tokens
for pos in self._position_bins:
    self.token2id[ 'Position_{}'.format(pos) ] = self.n_tokens
    self.n_tokens += 1

# Bar
self.token_type_base['Bar'] = self.n_tokens
self.token2id[ 'Bar' ] = self.n_tokens
self.n_tokens += 1

# Theme related tokens===== ...
# Phrase annotation (not used in our final paper)
self.token_type_base['Phrase'] = self.n_tokens
self.token2id[ 'Phrase_Start' ] = self.n_tokens
self.n_tokens += 1

self.token2id[ 'Phrase_End' ] = self.n_tokens
self.n_tokens += 1

# Theme annotation
self.token_type_base['Theme'] = self.n_tokens
self.token2id[ 'Theme_Start' ] = self.n_tokens
self.n_tokens += 1

self.token2id[ 'Theme_End' ] = self.n_tokens
self.n_tokens += 1

for w , v in self.token2id.items():
    self.id2token[v] = w

self.n_tokens = len(self.token2id)

def getPitch(self,input_event):
    """Return corresponding note pitch
    if input_event is not a note, it returns -1

    Args:
        input_event (str or int): REMI Event Name or vocab ID
    """
    if isinstance(input_event,int):
        input_event = self.id2token[input_event]
    elif isinstance(input_event,str):
        pass
    else:
        try:
            input_event = int(input_event)
            input_event = self.id2token[input_event]
        except:
            raise TypeError("input_event should be int or str, input_event={}, type={}".for ...

    if not input_event.startswith("Note-On"):
        return -1

    assert int(input_event.split("_")[1]) >=1 and int(input_event.split("_")[1]) <=127
    return int(input_event.split("_")[1])

def midi2REMI(self,midi_path,quantize=True,trim_intro = True,trim_outro=True,include_bridge ...
    """convert midi file to token representation

```

Args:

 midi_path (str): the path of input midi file
 trim_intro (bool, optional): ignore the intro part of the song. Defaults to True.
 trim_outro (bool, optional): ignore the outro part of the song. Defaults to True.
 include_bridge (bool, optional): ignore the intro part of the song. Defaults to Fal ...
 extend_theme (bool, optional): extend the theme region to at least MIN_MEL_NOTES=8 ...
 bar_first (bool, optional): place Bar token in front of Theme-Start token. Defaults ...
 theme_annotations (bool, optional): including theme-related tokens. Defaults to Tru ...
 verbose (bool, optional): print some message. Defaults to False.

Returns:

 list: sequence of tokens

"""

MIN_MEL_NOTES = 8

mid_obj = mid_parser.MidiFile(midi_path)

calculate the min step (in ticks) for REMI representation

min_step = mid_obj.ticks_per_beat * 4 / 16

quantize

if quantize:

 for i in range(len(mid_obj.instruments)):

 for n in range(len(mid_obj.instruments[i].notes)):

 mid_obj.instruments[i].notes[n].start = int(int(mid_obj.instruments[i].no ...

 mid_obj.instruments[i].notes[n].end = int(int(mid_obj.instruments[i].note ...

if theme_annotations:

 # select theme info track

 theme_boundary_track = list(filter(lambda x: x.name=="theme info track",mid_obj.in ...

 assert len(theme_boundary_track) == 1

parsing notes in each tracks (ignore BRIDGE)

notesAndtempos = []

mid_obj.instruments[0].notes = sorted(mid_obj.instruments[0].notes,key=lambda x: x.st ...

add notes

melody_start = sorted(mid_obj.instruments[0].notes,key=lambda x: x.start)[0].start

melody_end = sorted(mid_obj.instruments[0].notes,key=lambda x: x.start)[-1].end

for i in range(3):

 if not include_bridge and mid_obj.instruments[i].name == "BRIDGE":

 continue

 if mid_obj.instruments[i].name == "Theme info track":

 continue

 notes = mid_obj.instruments[i].notes

 for n in notes:

 # assert (trim_intro and n.start>=melody_start or not trim_intro)

 if trim_intro and n.start>=melody_start or not trim_intro:

 if trim_outro and n.start<=melody_end or not trim_outro:

 notesAndtempos.append({

 "priority" : i+1,

 "priority_1" : n.pitch,

 "start_tick" : n.start,

 "obj_type" : "Note-{}".format(mid_obj.instruments[i].name),

 "obj" : n

 })

add tempos

for t in mid_obj.tempo_changes:

 # assert (trim_intro and t.time>=melody_start or not trim_intro)

 if trim_intro and t.time>=melody_start or not trim_intro or trim_intro:

 if trim_outro and t.time<=melody_end or not trim_outro:

 notesAndtempos.append({

 "priority" : 0,

 "priority_1" : 0,

 "start_tick" : t.time,

 "obj_type" : "Tempo",

 "obj" : t

 })

if trim_intro and len([x for x in mid_obj.tempo_changes if x.time==melody_start]) == 0 ...

```

t = [x for x in sorted(midi_obj.tempo_changes, key= lambda z: z.time) if x.time <= me ...
if not len(t) == 0:
    t = t[-1]
    notesAndtempos.append({
        "priority" : 0,
        "priority_1" : 0,
        "start_tick" : melody_start,
        "obj_type" : "Tempo",
        "obj" : t
    })
notesAndtempos = sorted(notesAndtempos, key= lambda x: (x["start_tick"], x["priority"], -x[ ...

if theme_annotations:
    theme_boundary_track = theme_boundary_track[0]
    theme_boundary_pitch = min([x.pitch for x in theme_boundary_track.notes])
    theme_boundaries = [ [x.start, x.end] for x in theme_boundary_track.notes if x.pitch ...
    assert not len(theme_boundaries) == 0
    if verbose:
        print(theme_boundaries)

if extend_theme:
    # extend theme region 8-9
    for b_i, b in enumerate(theme_boundaries[:-1]):

        melody_notes = [x for x in midi_obj.instruments[0].notes if x.start >= b[0] and ...
        cur_bound = 0
        for x in melody_notes:
            if x.start < b[1]:
                cur_bound += 1
            else:
                break
        if cur_bound + 1 >= MIN_MEL_NOTES :
            continue
        # try to extend
        extend_idx = min(MIN_MEL_NOTES, len(melody_notes)) - 1
        theme_boundaries[b_i][1] = melody_notes[extend_idx].end


b_i = 0
in_theme = False
# group
bar_segments = []
bar_ticks = midi_obj.ticks_per_beat * 4
if verbose:
    print("Bar tick length: {}".format(bar_ticks))

for bar_start_tick in range(0, notesAndtempos[-1]["start_tick"], bar_ticks):
    if verbose:
        print("Bar {} at tick: {}".format(bar_start_tick // bar_ticks, bar_start_tick))
    bar_end_tick = bar_start_tick + bar_ticks
    current_bar = []
    bar_objs = list(filter(lambda x: x["start_tick"] >= bar_start_tick and x["start_tick" ...
    bar_objs.insert(0, {"start_tick": -1})

    if not bar_first:
        if theme_annotations and not in_theme and theme_boundaries[b_i][0] == bar_sta ...
            current_bar.append("Theme_Start")
            in_theme = True
            if verbose:
                print("Theme start")

    if not in_theme and trim_intro and bar_start_tick + bar_ticks < melody_start:
        if verbose:

```

```

        print("into trimmed")
        continue

    current_bar.append("Bar")
else:
    if not in_theme and trim_intro and bar_start_tick+bar_ticks < melody_start:
        if verbose:
            print("into trimmed")
            continue

    current_bar.append("Bar")

    if theme_annotations and not in_theme and theme_boundaries[b_i][0] == bar_start_tick:
        current_bar.append("Theme_Start")
        in_theme = True
        if verbose:
            print("Theme start")

for i,obj in enumerate(bar_objs):
    if obj["start_tick"]==-1 : continue
    if not obj["start_tick"] == bar_objs[i-1]["start_tick"]:
        # insert Position Event
        pos = (obj["start_tick"] - bar_start_tick) / midi_obj.ticks_per_beat * self ...
        pos_index = np.argmin(abs(pos - self._position_bins)) # use the closest pos ...
        pos = self._position_bins[pos_index]
        current_bar.append("Position-{}".format(pos))

    if obj["obj_type"].startswith("Note"):
        track_name = obj["obj_type"].split('-')[1].upper()
        # add pitch
        current_bar.append("Note-On-{}-{}".format(track_name,obj["obj"].pitch))
        # add duration
        dur = (obj["obj"].end - obj["obj"].start) / midi_obj.ticks_per_beat * self ...
        dur_index = np.argmin(abs(dur - self._duration_bins)) # use the closest pos ...
        dur = self._duration_bins[dur_index]
        current_bar.append("Note-Duration-{}-{}".format(track_name,dur))
        # add velocity
        vel_index = np.argmin(abs(obj["obj"].velocity - self._velocity_bins)) # use ...
        vel = self._velocity_bins[vel_index]
        current_bar.append("Note-Velocity-{}-{}".format(track_name,vel))
    elif obj["obj_type"].startswith("Tempo"):
        # tempo
        tmp_index = np.argmin(abs(obj["obj"].tempo - self._tempo_bins)) # use the c ...
        tmp = self._tempo_bins[tmp_index]
        current_bar.append(obj["obj_type"] + "-{}".format(tmp))
    else:
        # theme start end
        current_bar.append(obj["obj_type"])

    if theme_annotations and in_theme and theme_boundaries[b_i][1] == bar_start_tick:
        current_bar.append("Theme_End")
        in_theme = False
        if verbose:
            print("Theme End")
        if not b_i == len(theme_boundaries) - 1:
            b_i += 1
    bar_segments.extend(current_bar)

output_ids = [self.token2id[x] for x in bar_segments]

return output_ids

def preprocessREMI(self, remi_sequence, always_include=False, max_seq_len=512, strict=True, verb ...
    """Preprocess token sequence

    slicing the sequence for training our models
    Args:

```

remi_sequence (List): the music token sequence
 always_include (Bool): selected the data including either Theme-Start or Theme-End
 max_seq_len (Int): maximum sequence length for each data
 strict (Bool): the returning sequence should always include Theme-Start

Return:

```

{
    "src" : &lt;corressponding theme condition&gt;;
    "src_theme_binary_msk" : &lt;corressponding theme condition's theme msk&gt;;
    "tgt_segments" : &lt;list of target sequences&gt;;
    "tgt_segments_theme_binary_msk" : &lt;list of target sequences theme msk&gt;;
}
"""
theme_binary_msk = []
in_theme = False
src = []
src_theme_binary_msk = []
for r in remi_sequence:
    if self.id2token[r] == "Theme_Start":
        in_theme = True
    elif self.id2token[r] == "Theme_End":
        in_theme = False
    theme_binary_msk.append(int(in_theme))

for i in range(1, len(theme_binary_msk)):
    theme_binary_msk[i] = theme_binary_msk[i-1]*theme_binary_msk[i] + theme_binary_msk[ ...

start_first_theme = remi_sequence.index(self.token2id["Theme_Start"])
end_first_theme = remi_sequence.index(self.token2id["Theme_End"])
src = remi_sequence[start_first_theme:end_first_theme+1]
src_theme_binary_msk = theme_binary_msk[start_first_theme:end_first_theme+1]

tgt_segments = []
tgt_segments_theme_msk = []

s = 0
if strict:
    theme_start_pos = [i for i in range(len(remi_sequence)) if remi_sequence[i] == self ...
    for t in theme_start_pos:
        tgt_segments.append(remi_sequence[t:t+max_seq_len+1])
        tgt_segments_theme_msk.append(theme_binary_msk[t:t+max_seq_len+1])
else:
    total_s = math.ceil(len(remi_sequence) / max_seq_len)
    for x in range(0, len(remi_sequence), max_seq_len):
        if always_include:
            # if self.token2id["Theme_Start"] in remi_sequence[x:x+max_seq_len+1]:
            if self.token2id["Theme_Start"] in remi_sequence[x:x+max_seq_len+1] or self ...
                s += 1
                tgt_segments.append(remi_sequence[x:x+max_seq_len+1])
                tgt_segments_theme_msk.append(theme_binary_msk[x:x+max_seq_len+1])
        else:
            tgt_segments.append(remi_sequence[x:x+max_seq_len+1])
            tgt_segments_theme_msk.append(theme_binary_msk[x:x+max_seq_len+1])

    if verbose and always_include:
        print("Include Theme Start {}/{}".format(s, total_s))

return {
    "src" : src,
    "src_theme_binary_msk" : src_theme_binary_msk,
    "tgt_segments" : tgt_segments,
    "tgt_segments_theme_binary_msk" : tgt_segments_theme_msk
}

```

```
def REMIID2midi(self, event_ids, midi_path, verbose = False):
```



```

"""convert tokens to midi file
The output midi file will contains 3 tracks:
    MELODY : melodt notes
    PIANO : accompaniment notes
    Theme info track : notes indicating theme region (using note start tick and end tic ...
Args:
    event_ids (list): sequence of tokens
    midi_path (str): the output midi file path
    verbose (bool, optional): print some message. Defaults to False.
"""

# create midi file
new_mido_obj = mid_parser.MidiFile()
new_mido_obj.ticks_per_beat = 120

# create tracks
music_tracks = {}
music_tracks["MELODY"] = ct.Instrument(program=0, is_drum=False, name='MELODY')
music_tracks["PIANO"] = ct.Instrument(program=0, is_drum=False, name='PIANO')
music_tracks["Theme info track"] = ct.Instrument(program=0, is_drum=False, name='Theme ...

# all our generated music are 4/4
new_mido_obj.time_signature_changes.append(miditoolkit.TimeSignature(4,4,0))

ticks_per_step = new_mido_obj.ticks_per_beat / self.q_beat

# convert tokens from id to string
events = []
for x in event_ids:
    events.append(self.id2token[x])

# parsing tokens
last_tick = 0
current_bar_anchor = 0
current_theme_boundary = []
motif_label_segs = []
idx = 0
first_bar = True
while(idx < len(events)):
    if events[idx] == "Bar":
        if first_bar:
            current_bar_anchor = 0
            first_bar = False
        else:
            current_bar_anchor += new_mido_obj.ticks_per_beat * 4
        idx+=1
    elif events[idx].startswith("Position"):
        pos = int(events[idx].split('_')[1])
        last_tick = pos * ticks_per_step + current_bar_anchor
        idx += 1
    elif events[idx].startswith("Tempo"):
        tmp = pos = int(events[idx].split('_')[1])
        new_mido_obj.tempo_changes.append(ct.TempoChange(
            tempo=int(tmp),
            time=int(last_tick)
        ))
        idx += 1
    elif events[idx].startswith("Note"):
        track_name = events[idx].split("_")[0].split("-")[2]
        assert track_name in music_tracks
        assert events[idx].startswith("Note-On")
        assert events[idx+1].startswith("Note-Duration")
        assert events[idx+2].startswith("Note-Velocity")

        new_note = miditoolkit.Note(
            velocity=int(events[idx+2].split("_")[1]),

```

```

        pitch=int(events[idx].split("_")[1]),
        start=int(last_tick),
        end=int(int(events[idx+1].split('_')[1]) * ticks_per_step) + int(last_tick)
    )
    music_tracks[track_name].notes.append(new_note)
    idx += 3
elif events[idx] == "Theme_Start":
    assert len(current_theme_boundary) == 0
    current_theme_boundary.append(last_tick)
    idx += 1
elif events[idx] == "Theme_End":
    assert len(current_theme_boundary) == 1
    current_theme_boundary.append(last_tick)
    motif_label_segs.append(current_theme_boundary)
    music_tracks["Theme info track"].notes.append(
        miditoolkit.Note(
            velocity=1,
            pitch=1,
            start=int(current_theme_boundary[0]),
            end=int(current_theme_boundary[1])
        )
    )
    current_theme_boundary = []
    idx += 1

# add tracks to midi file
new_mido_obj.instruments.extend([music_tracks[ins] for ins in music_tracks])

if verbose:
    print("Saving midi to {}".format(midi_path))

# save to disk
new_mido_obj.dump(midi_path)

def __str__(self):
    """return all tokens

    Returns:
        str: string of all tokens
    """
    ret = ""
    for w,i in self.token2id.items():
        ret = ret + "{} : {}\n".format(w,i)

    for i,w in self.id2token.items():
        ret = ret + "{} : {}\n".format(i,w)

    ret += "\nTotal events {}".format(len(self.id2token))

    return ret

def __repr__(self):
    """return string all token

    Returns:
        str: string of sll tokens
    """
    return self.__str__()

if __name__ == '__main__':
    # print all tokens
    myvocab = Vocab()

    print(myvocab)

```

7. Dataset & Data Loading — preprocess/music_data.py

Provides **MusicDataset** (PyTorch Dataset) and **getMusicDataset()** factory. Each sample includes source (theme), target, and corresponding padding / theme masks. Includes pitch augmentation for data diversity.

Sample Dictionary Keys

Key	Shape	Description
src	[max_src_len]	Theme token IDs (padded)
src_msk	[max_src_len]	True = padding position
src_theme_msk	[max_src_len]	Cumulative theme position index for src
tgt	[max_tgt_len]	Full-song token IDs (padded)
tgt_msk	[max_tgt_len]	True = padding position
tgt_theme_msk	[max_tgt_len]	Cumulative theme position index for tgt

Source Code

```
"""Music Dataset for theme-based transformer

Author: Ian Shih
Email: yjshih23@gmail.com
Date: 2021/11/03

"""
import torch
from torch.utils.data.dataset import Dataset
import sys, pickle
import numpy as np
from glob import glob
from copy import deepcopy

import preprocess.vocab

# create vocab
myvocab = preprocess.vocab.Vocab()

import random

class MusicDataset(Dataset):
    def __init__(self, data, max_seq_len, vocab = None, aug=None):
        self.vocab = myvocab if vocab == None else vocab
        self.event2id = self.vocab.token2id
        self.id2event = self.vocab.id2token

        self.data = data

        self.max_seq_len = max_seq_len

        self.ep_start_pitchaug = 0

        self.pitchaug_range = aug
        self.final_training_data = []

        self.constants = {
            "max_src_len" : max([ len(x["src"]) for x in self.data]),
            "max_tgt_len" : min(max_seq_len, max([ len(x["tgt"]) for x in self.data])) + 1
        }
        print(self.constants)
```

```

def data_pitch_augment(self,src_seq,tgt_seq):
    """pitch shift for data augmentation

    Args:
        src_seq (list): src sequence
        tgt_seq (list): tgt sequence

    Returns:
        tuple: augmented src sequence, augmented target sequence
    """
    # pitch augement

    # get all pitch tokens
    all_pitches = [ self.vocab.getPitch(x) for x in src_seq] + [self.vocab.getPitch(x) for ...
    all_pitches = [x for x in all_pitches if x > 0]
    if len(all_pitches) == 0 :
        return src_seq,tgt_seq

    pitch_offsets = np.random.randint(1-min(all_pitches),127 - max(all_pitches), size=1)
    pitch_offset = pitch_offsets[0]

    aug_src_phrase = deepcopy(src_seq)
    aug_tgt_phrase = deepcopy(tgt_seq)
    for t in range(len(aug_src_phrase)):
        if self.vocab.getPitch(aug_src_phrase[t]) > 0:
            aug_src_phrase[t] += pitch_offset
            assert self.vocab.getPitch(aug_src_phrase[t]) > 0

    for t in range(len(aug_tgt_phrase)):
        if self.vocab.getPitch(aug_tgt_phrase[t]) > 0:
            aug_tgt_phrase[t] += pitch_offset
            assert self.vocab.getPitch(aug_tgt_phrase[t]) > 0

    return aug_src_phrase, aug_tgt_phrase

def __getitem__(self, index):
    """return data given index

    Args:
        index (int): the index fo data

    Returns:
        obj: {
            "src"           : <src sequence>;
            "src_msk"       : <src sequence padding mask>;,
            "src_theme_msk" : <src sequence theme mask>;,
            "tgt"           : <target sequence>;,
            "tgt_msk"       : <target sequence padding mask>;,
            "tgt_theme_msk" : <target sequence theme mask>;,
        }
    """

    # pitch augment
    src, tgt = self.data_pitch_augment(src_seq=self.data[index]["src"],tgt_seq=self.data[in ...

    src_theme_msk = self.data[index]["src_theme_msk"]
    tgt_theme_msk = self.data[index]["tgt_theme_msk"]

    src_theme_msk = list(map(int,src_theme_msk))
    tgt_theme_msk = list(map(int,tgt_theme_msk))

    # padding
    src_msk = [0]* len(src) + [1] * (self.constants["max_src_len"] - len(src))
    src_theme_msk.extend([0]*(self.constants["max_src_len"] - len(src)))
    src.extend([0]*(self.constants["max_src_len"] - len(src)))
    tgt_msk = []
    if len(tgt) > self.constants["max_tgt_len"]:

```

```

        print("Should not be here")
        assert True
        tgt = tgt[:self.constants["max_tgt_len"]]
        tgt_theme_msk = tgt_theme_msk[:self.constants["max_tgt_len"]]
        tgt_msk = [0] * len(tgt)
    else:
        tgt_msk = [0]* len(tgt) + [1] * (self.constants["max_tgt_len"] - len(tgt))
        tgt_theme_msk.extend([0]*(self.constants["max_tgt_len"] - len(tgt)))
        tgt.extend([0]*(self.constants["max_tgt_len"] - len(tgt)))

    current_entry = {
        "src"          : src,
        "src_msk"       : src_msk,
        "src_theme_msk" : src_theme_msk,
        "tgt"           : tgt,
        "tgt_msk"       : tgt_msk,
        "tgt_theme_msk" : tgt_theme_msk,
    }
    assert(len(src) == self.constants["max_src_len"])
    assert(len(tgt) == self.constants["max_tgt_len"])
    assert(len(src_msk) == self.constants["max_src_len"])
    assert(len(tgt_msk) == self.constants["max_tgt_len"])
    assert(len(src_theme_msk) == self.constants["max_src_len"])
    assert(len(tgt_theme_msk) == self.constants["max_tgt_len"])

    return {key: torch.tensor(value) for key, value in current_entry.items()}

def __len__(self):
    return len(self.data)

# def set_epoch(self,ep):

def getMusicDataset(pk1_path,args,vocab):
    """load data from pk1 file and return torch dataset

    Args:
        pk1_path (str): data pk1 file path
        args (obj): all args from argparser
        vocab (Vocab): vocab instance

    Returns:
        torch.util.data.dataset: the dataset insrance
    """
    with open(pk1_path,"rb") as f:
        train_data = pickle.load(f)

    dataset = MusicDataset(data=train_data,max_seq_len=args.max_len,vocab=vocab)

    return dataset

if __name__ == '__main__':
    pass

```

8. Data Preprocessing — preprocess/prepareDataPkl.py

Batch-converts theme-annotated POP909 MIDI files into training PKL data. Steps: (1) MIDI → REMI tokens via vocab.midi2REMI, (2) slice into windows via vocab.preprocessREMI, (3) flatten all segments into one training_data list, (4) dump to PKL.

Source Code

```
import vocab
import numpy as np
import glob
import os
import pickle

myvocab = vocab.Vocab()

# Check the paths for your own case
# the theme annotated midi files
MIDI_FILES = "./pop909_midis/*.mid"

# the tokens converted from theme annotated midi files
MIDI_FILES_PKLS_DIR = "./pop909_midi_pkls"

# the output training data
OUTPUT_DATA_PKL = "../data_pkl/train_seg2_512.pkl"

all_mids = sorted(glob.glob(MIDI_FILES))

for _midiFile in all_mids:
    # convert midi files to token representation and save as .pkl file
    output_pkl_fp = _midiFile.replace(".mid", ".pkl")
    remi_seq = myvocab.midi2REMI(_midiFile, include_bridge=False, bar_first=False, verbose=False)
    ret = myvocab.preprocessREMI(remi_seq, always_include=True, max_seq_len=512, verbose=True)
    pickle.dump(ret, open(output_pkl_fp, 'wb'), protocol=pickle.HIGHEST_PROTOCOL)

# collect all .pkl files and generate .pkl file for training/testing
al_pkls = sorted(glob.glob(os.path.join(MIDI_FILES_PKLS_DIR, "*.pkl")))

training_data = []

for i, fn in enumerate(al_pkls):
    print(">>>[{} / {}][train] Now processing {}".format(i+1, len(al_pkls), os.path.split(fn)[-1]))
    with open(fn, "rb") as f:
        data = pickle.load(f)
        src = data["src"]
        src_theme_binary_msk = data["src_theme_binary_msk"]

        for i_tgt, tgt in enumerate(data["tgt_segments"]):
            training_data.append({"src": src, "tgt": tgt,
                                  "tgt_theme_msk": data["tgt_segments_theme_binary_msk"][i_tgt],
                                  "src_theme_msk": src_theme_binary_msk})

# dump to pkl file for training
pickle.dump(training_data, open(OUTPUT_DATA_PKL, 'wb'), protocol=pickle.HIGHEST_PROTOCOL)
```

9. Training Script — train.py

Loads data, initialises the model, runs the training loop for up to 15,000 epochs, and saves checkpoints every 10 epochs.

Training Configuration

Setting	Value
Model hidden dim	256
Encoder layers	6
Decoder pattern	[0, 0, 0, 1, 1, 1] (3 plain + 3 GPA)
Optimizer	Adam lr=0.0002
Scheduler	CosineAnnealingLR T_max=3,200,000 eta_min=1e-5
Warmup steps	400
Gradient clip	3.0
Batch size	8
Loss	CrossEntropyLoss (ignore_index=0)
Max epochs	15,000
Checkpoint interval	Every 10 epochs

Source Code

```
"""Theme Transformer Training Code

usage: inference.py [-h] [--model_path MODEL_PATH] [--theme THEME]
                  [--seq_length SEQ_LENGTH] [--seed SEED]
                  [--out_midi OUT_MIDI] [--cuda] [--max_len MAX_LEN]
                  [--temp TEMP]
  --model_path MODEL_PATH  model file
  --theme THEME            theme file
  --seq_length SEQ_LENGTH  generated seq length
  --seed SEED              random seed (set to -1 to use random seed) (change different if the ...
  --out_midi OUT_MIDI      output midi file
  --cuda                  use CUDA
  --max_len MAX_LEN        number of tokens to predict
  --temp TEMP              temperature

  Author: Ian Shih
  Email: yjshih23@gmail.com
  Date: 2021/11/03

"""
device_str = 'cuda:0'
import shutil
from torch.utils.data import DataLoader
import numpy as np
import torch
import torch.optim

from mymodel import myLM

from preprocess.music_data import getMusicDataset

from preprocess.vocab import Vocab
```

```

from parse_arg import *

import time
import os

import logger

from randomness import set_global_random_seed

# Set the random seed manually for reproducibility.
set_global_random_seed(args.seed)

# create vocab
myvocab = Vocab()

# create directory for training purpose
os.makedirs("./ckpts", exist_ok=True)
os.makedirs("./logs", exist_ok=True)

# create work directory
while(1):
    exp_name = input("Enter exp name : ")
    if os.path.exists(os.path.join("./ckpts", exp_name)):
        ans = input("work dir exists! overwrite? [Y/N]:")
        if ans.lower() == "y":
            break
    else:
        break

os.makedirs(os.path.join(
    "./ckpts/", exp_name), exist_ok=True)
os.makedirs(os.path.join("./ckpts/",
    exp_name, "script"), exist_ok=True)
os.makedirs(os.path.join("./ckpts/",
    exp_name, "script", "preprocess"), exist_ok=True)
os.makedirs(os.path.join("./ckpts/",
    exp_name, "log"), exist_ok=True)

checkpoint_folder = "./ckpts/{}".format(
    exp_name)
# copy scripts
file_to_save = ['train.py', 'inference.py', 'myTransformer.py', 'randomness.py',
    'parse_arg.py', 'mymodel.py', 'preprocess/vocab.py', 'preprocess/music_data.py' ...]
for x in file_to_save:
    shutil.copyfile(x, os.path.join(checkpoint_folder, "script", x))

# create logger for log
mylogger = logger.logger(filepath=os.path.join(
    checkpoint_folder, "log/log_{}.txt".format(exp_name)), overwrite=True)
if os.path.exists("logs/log_{}.txt".format(exp_name)):
    os.remove("logs/log_{}.txt".format(exp_name))
os.link(mylogger.filepath, "logs/log_{}.txt".format(exp_name))
mylogger.log("Exp_dir : {}".format(checkpoint_folder))
mylogger.log("Exp_Name : {}".format(exp_name))

# devices
device = torch.device( device_str if args.cuda else 'cpu')
device_cpu = torch.device('cpu')

# dataset
train_dataset = getMusicDataset(pk1_path="./data_pk1/train_seg2_512.pk1",
    args=args,
    vocab=myvocab)

val_dataset = getMusicDataset(pk1_path="./data_pk1/val_seg2_512.pk1",

```



```

        args=args,
        vocab=myvocab)

train_loader = DataLoader(dataset=train_dataset,
                           batch_size=args.batch_size,
                           shuffle=True,
                           num_workers=4)

val_loader = DataLoader(dataset=val_dataset,
                        batch_size=2,
                        shuffle=False,
                        num_workers=4)

# define model
model = myLM(myvocab.n_tokens, d_model=256,num_encoder_layers=6,xorpattern=[0,0,0,1,1,1])

mylogger.log("Model hidden dim : {}".format(model.d_model))
mylogger.log("Encoder Layers {}".format(model.num_encoder_layers))
mylogger.log("Decoder Layers {}".format(len(model.xorpattern)))
mylogger.log("Decoder Pattern {}".format(model.xorpattern))
mylogger.log("Batch size {}".format(args.batch_size))
mylogger.log("lr : {}".format(args.lr))
# optimizer
# adam
optimizer = torch.optim.Adam(model.parameters(), lr=args.lr)

# scheduler
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=args.max_step,eta_min=args.lr_min)

if not args.restart_point == '':
    # restart from checkpoint
    mylogger.log("Restart from {}".format(args.restart_point))
    model.load_state_dict(torch.load(args.restart_point,map_location=device_str))
    mylogger.log("model loaded")
    optimizer.load_state_dict(torch.load(args.restart_point.replace('model_', 'optimizer_'),map_ ...
    mylogger.log("optimizer loaded")
    scheduler.load_state_dict(torch.load(args.restart_point.replace('model_', 'scheduler_'),map_ ...
    mylogger.log("scheduler loaded")

    for state in optimizer.state.values():
        for k, v in state.items():
            if torch.is_tensor(v):
                state[k] = v.to(device)

# loss
criterion = torch.nn.CrossEntropyLoss(ignore_index=0)

mylogger.log("Using device : {}".format(logger.getCyan(device)))

train_step = 0

def train(epoch_num):
    """train the model

    Args:
        epoch_num (int): epoch number
    """
    global train_step

    model.train()
    start_time = time.time()
    total_loss = 0
    total_acc = 0
    steps = 0
    for batch_idx, data in enumerate(train_loader):

```

```

print("Epoch {} Step [{}/{}] ".format(epoch_num,
    batch_idx, len(train_loader)), end='')

data = {key: value.to(device) for key, value in data.items()}

optimizer.zero_grad()

data["src_msk"] = data["src_msk"].bool()
data["tgt_msk"] = data["tgt_msk"].bool()

tgt_input_msk = data["tgt_msk"][:, :-1]
tgt_output_msk = data["tgt_msk"][:, 1:]

data["src"] = data["src"].permute(1, 0)
data["tgt"] = data["tgt"].permute(1, 0)
data["tgt_theme_msk"] = data["tgt_theme_msk"].permute(1, 0)

fullsong_input = data["tgt"][:-1, :]
fullsong_output = data["tgt"][1:, :]

att_msk = model.transformer_model.generate_square_subsequent_mask(
    fullsong_input.shape[0]).to(device)

mem_msk = None

output = model(
    src=data["src"],
    tgt=fullsong_input,
    tgt_mask=att_msk,
    tgt_label=data["tgt_theme_msk"][:-1, :],
    src_key_padding_mask=data["src_msk"],
    tgt_key_padding_mask=tgt_input_msk,
    memory_mask=mem_msk
)

loss = criterion(output.view(-1, myvocab.n_tokens),
    fullsong_output.reshape(-1))

predict = output.view(-1, myvocab.n_tokens).argmax(dim=-1)

correct = predict.eq(fullsong_output.reshape(-1))

correct = torch.sum(
    correct * (~tgt_output_msk).reshape(-1).float()).item()

correct = correct / \
    torch.sum((~tgt_output_msk).reshape(-1).float()).item()

total_acc += correct

print("Acc : {:.2f} ".format(correct), end="")

loss.backward()

torch.nn.utils.clip_grad_norm_(model.parameters(), args.clip)

optimizer.step()

if train_step < args.warmup_step:
    curr_lr = args.lr * train_step / args.warmup_step
    optimizer.param_groups[0]['lr'] = curr_lr
else:
    scheduler.step()

total_loss += loss.item()

```

```

        print("Loss : {:.2f} lr:{:.4f} ".format(
            loss.item(), optimizer.param_groups[0]['lr']), end='\r')

        steps += 1
        train_step += 1

    mylogger.log("Epoch {} lr:{:.4f} train_acc : {:.2f} train_loss : {:.2f} time:{:.2f} ".form ...
    ...

def evaluate(epoch_num):
    """evaluate validation set

    Args:
        epoch_num (int): epoch number
    """
    model.eval()
    start_time = time.time()
    total_loss = 0
    total_acc = 0
    steps = 0
    with torch.no_grad():
        for batch_idx, data in enumerate(val_loader):
            # print("Epoch {} Step {}/{}".format( epoch_num, batch_idx, len(val_loader)), end='')
            optimizer.zero_grad()

            data = {key: value.to(device) for key, value in data.items()}

            data["src_msk"] = data["src_msk"].bool()
            data["tgt_msk"] = data["tgt_msk"].bool()

            tgt_input_msk = data["tgt_msk"][:, :-1]
            tgt_output_msk = data["tgt_msk"][:, 1:]

            data["src"] = data["src"].permute(1, 0)
            data["tgt"] = data["tgt"].permute(1, 0)
            data["tgt_theme_msk"] = data["tgt_theme_msk"].permute(1, 0)

            fullsong_input = data["tgt"][:, -1, :]
            fullsong_output = data["tgt"][1:, :]

            att_msk = model.transformer_model.generate_square_subsequent_mask(
                fullsong_input.shape[0]).to(device)

            mem_msk = None

            output = model(
                src=data["src"],
                tgt=fullsong_input,
                tgt_mask=att_msk,
                tgt_label=data["tgt_theme_msk"][:, -1, :],
                src_key_padding_mask=data["src_msk"],
                tgt_key_padding_mask=tgt_input_msk,
                memory_mask=mem_msk
            )

            loss = criterion(output.view(-1, myvocab.n_tokens),
                             fullsong_output.reshape(-1))

            predict = output.view(-1, myvocab.n_tokens).argmax(dim=-1)

            correct = predict.eq(fullsong_output.reshape(-1))

            correct = torch.sum(
                correct * (~tgt_output_msk).reshape(-1).float()).item()

            correct = correct / \
                torch.sum((~tgt_output_msk).reshape(-1).float()).item()

```

```

        total_acc += correct

        total_loss += loss.item()

        steps += 1

    mylogger.log("val_acc: {:.2f} val_loss : {:.2f}".format(
        total_acc/steps, total_loss/steps))

start_epoch = 0

if not args.restart_point == '':
    start_epoch = int(args.restart_point.split('_')[-1].split('.')[0][2:]) + 1
    mylogger.log("starting from epoch {}".format(start_epoch))

max_epoch = 15000
mylogger.log("max epoch :{}".format(max_epoch))
for i in range(start_epoch,max_epoch):
    model.to(device)
    train(i)
    evalulate(i)
    model.to(device_cpu)

    if i % 10 == 0:
        # save state dicts
        torch.save(model.state_dict(), os.path.join(
            checkpoint_folder, "model_ep{}.pt".format(i)))
        torch.save(optimizer.state_dict(), os.path.join(
            checkpoint_folder, "optimizer_ep{}.pt".format(i)))
        torch.save(scheduler.state_dict(), os.path.join(
            checkpoint_folder, "scheduler_ep{}.pt".format(i)))

```

10. Inference Script — inference.py

Generates a full multi-track MIDI piece conditioned on a given theme MIDI file. Uses nucleus sampling with temperature scaling. Includes grammar checking to ensure token sequence validity.

CLI Arguments

Argument	Default	Description
--model_path	trained_model/model_ep2311.pt	Path to model checkpoint
--theme	(required)	Input theme MIDI file
--out_midi	output.mid	Output MIDI file path
--nbars	32	Number of bars to generate
--max_len	512	Rolling context window size (tokens)
--temp	1.2	Temperature for sampling
--seed	-1	Random seed (-1 = random)
--cuda	False	Use GPU

Grammar Checking Rules

- Theme_Start / Theme_End must be followed immediately by Bar.
- Note-On must be followed by Note-Duration of the same track.
- Note-Duration must be followed by Note-Velocity of the same track.
- Tempo / Note tokens require a Position token to be set first.
- Position tokens must strictly increase within a bar.
- Theme_Start / Theme_End must appear in alternating order.

Source Code

```
"""Theme Transformer Inferencing Code

usage: inference.py [-h] [--model_path MODEL_PATH] --theme THEME
                  [--seq_length SEQ_LENGTH] [--seed SEED]
                  [--out_midi OUT_MIDI] [--cuda] [--max_len MAX_LEN]
                  [--temp TEMP] [--nbars NBARS]
--model_path MODEL_PATH  model file
--theme THEME            theme file
--seq_length SEQ_LENGTH  generated seq length
--seed SEED              random seed (set to -1 to use random seed) (change different if the ...
--out_midi OUT_MIDI      output midi file
--cuda                  use CUDA
--max_len MAX_LEN        number of tokens to predict
--temp TEMP              temperature
--nbars NBARS            number of bars to generate

Author: Ian Shih
Email: yjshih23@gmail.com
Date: 2021/11/03
"""

import argparse
import numpy as np
```

```

import torch
import torch.optim

from mymodel import myLM

from preprocess.music_data import MusicDataset

from preprocess.vocab import Vocab

import time
import os
import pickle
import json

from randomness import set_global_random_seed

parser = argparse.ArgumentParser()
parser.add_argument('--model_path', type=str, default='./trained_model/model_ep2311.pt',
                    help='model file')

parser.add_argument('--theme', type=str, required=True, default='./theme_files/874_theme.mid',
                    help='theme file')

parser.add_argument('--seq_length', type=str, default='',
                    help='generated seq length')

parser.add_argument('--seed', type=int, default=-1,
                    help='random seed (set to -1 to use random seed)')

parser.add_argument('--out_midi', type=str, default='output.mid',
                    help='output midi file')

parser.add_argument('--cuda', action='store_true',
                    help='use CUDA')

parser.add_argument('--max_len', type=int, default=512,
                    help='number of tokens to predict')

parser.add_argument('--temp', type=float, default=1.2,
                    help='temperature')

parser.add_argument('--nbars', type=float, default=32,
                    help='number of bars to generate')

args = parser.parse_args()

if not args.seed == -1:
    set_global_random_seed(args.seed)

# create vocab
myvocab = Vocab()

# devices
device = torch.device('cuda:0' if args.cuda else 'cpu')
device_cpu = torch.device('cpu')

# model definition
model = myLM(myvocab.n_tokens, d_model=256, num_encoder_layers=6, xorpattern=[0,0,0,1,1,1])

print("Loading model from {}".format(args.model_path))
model.load_state_dict(torch.load(args.model_path))
print("Using device {}".format(device))

def inference(n_bars, strategies, params, theme_seq, prompt=None):
    """inference function

    Args:

```

```

        n_bars (int): numbers of bar to generate
        strategies (dict): inferencing strategies
        params (dict): parameters for inferencing strategies
        theme_seq (list): given theme condition
        prompt (list, optional): initial tokens fed to the theme transformer. Defaults to None.

Returns:
    list: token sequence of generated music
"""
model.eval()
words = [[]]

word2event = myvocab.id2token

initial_flag, initial_cnt = True, 0
generated_bars = 0

fail_cnt = 0
subbeats_accumulate = 0

input_theme = torch.tensor(theme_seq)
input_theme = input_theme.reshape((-1,1))
input_theme = input_theme.to(device)

label_list = []

previous_labeled = False

last_theme_bar_idx = -1

new_motif_tmp_array = []

bar_count = 0
position_anchor = -1

with torch.no_grad():
    while bar_count < n_bars:
        print("events #{0} Generating Bars #{0}/{0}".format(len(words[0]),bar_count ,bar_count ...
        if fail_cnt:
            print ('failed iterations:', fail_cnt)

        if fail_cnt > 1024:
            print ('model stuck ...\nPlease change a seed sand inference again!')
            return words[0]

        # prepare input
        if initial_flag:
            if not prompt == None:
                # prompt given
                input_x = torch.tensor(prompt)
                words[0].extend(prompt)
                label_list = [0]*len(prompt)
                for i,x in enumerate(prompt):
                    if myvocab.id2token[x] == "Theme_Start":
                        previous_labeled = True
                    elif myvocab.id2token[x] == "Theme_End":
                        previous_labeled = False
                    if previous_labeled:
                        if i == 0:
                            label_list[i] = 1
                        else:
                            label_list[i] = label_list[i-1] + 1
                    if myvocab.id2token[x].startswith("Position"):
                        position_anchor = int(myvocab.id2token[x].split("_")[1])
                    if myvocab.id2token[x] == "Bar":
                        position_anchor = - 1
                        n_bars += 1

                label_input = torch.tensor(label_list)

```

```

else:
    # no prompt given
    input_x = torch.tensor([theme_seq[0]])
    label_list = [0]
    words[0].append(theme_seq[0])
    if myvocab.id2token[theme_seq[0]] == "Theme_Start":
        previous_labeled = True
    label_input = torch.tensor(label_list)

    initial_flag = False
else:
    input_x = torch.tensor(words[0][-args.max_len:])
    label_input = torch.tensor(label_list[-args.max_len:])

input_x = input_x.reshape((-1,1))
label_input = label_input.reshape((-1,1))

input_x_att_msk = model.transformer_model.generate_square_subsequent_mask(input_x.s ...
input_x = input_x.to(device)
label_input = label_input.to(device)
input_x_att_msk = input_x_att_msk.to(device)

logits = model(
    src=input_theme,
    tgt=input_x,
    tgt_label=label_input,
    tgt_mask = input_x_att_msk
)
logits = logits[-1:]
logits = torch.squeeze(logits)
logits = logits.cpu().numpy()

# temperature or not
if 'temperature' in strategies:
    probs = model.temperature(logits=logits, temperature=params['t'])
else:
    probs = model.temperature(logits=logits, temperature=1.)

# sampling
# word : the generated remi event
word = model.nucleus(probs=probs, p=params['p'])

print("Generated new remi word {}".format(myvocab.id2token[word]))
# skip padding
if word in [0]:
    fail_cnt += 1
    continue

# grammar checking =====

# check Theme_Start -> Bar
if 'Theme_Start' in word2event[words[0][-1]] and 'Bar' not in word2event[word]:
    fail_cnt += 1
    print(490)
    continue

# check Theme_End -> Bar
if 'Theme_End' in word2event[words[0][-1]] and 'Bar' not in word2event[word]:
    fail_cnt += 1
    print(490)
    continue

# check Note-On-[track] -> Note-Duration-[track]

```



```

if 'Note-On' in word2event[words[0][-1]] and 'Note-Duration' not in word2event[word]:
    fail_cnt += 1
    print(490)
    continue
if 'Note-On' in word2event[words[0][-1]] and 'Note-Duration' in word2event[word]:
    if not word2event[words[0][-1]].split("_")[0].split("-")[2] == word2event[word].split("_")[0].split("-")[2]:
        print("Note-On,Duration Track Inconsistency")
        continue

if 'Note-Duration' in word2event[word] and 'Note-On' not in word2event[words[0][-1]]:
    fail_cnt += 1
    print(490)
    continue
if 'Note-Duration' in word2event[word] and 'Note-On' in word2event[words[0][-1]]:
    if not word2event[words[0][-1]].split("_")[0].split("-")[2] == word2event[word].split("_")[0].split("-")[2]:
        print("Note-On,Duration Track Inconsistency")
        continue

# check Note-Duration-[track] -&gt; Note-Velocity-[track]
if 'Note-Duration' in word2event[words[0][-1]] and 'Note-Velocity' not in word2event[word]:
    fail_cnt += 1
    print(490)
    continue
if 'Note-Duration' in word2event[words[0][-1]] and 'Note-Velocity' in word2event[word]:
    if not word2event[words[0][-1]].split("_")[0].split("-")[2] == word2event[word].split("_")[0].split("-")[2]:
        print("Note-Duration,Velocity Track Inconsistency")
        continue

if 'Note-Velocity' in word2event[word] and 'Note-Duration' not in word2event[words[0][-1]]:
    fail_cnt += 1
    print(490)
    continue
if 'Note-Velocity' in word2event[word] and 'Note-Duration' in word2event[words[0][-1]]:
    if not word2event[words[0][-1]].split("_")[0].split("-")[2] == word2event[word].split("_")[0].split("-")[2]:
        print("Note-Duration,Velocity Track Inconsistency")
        continue

if word2event[word].startswith("Tempo") or word2event[word].startswith("Note"):
    if position_anchor == -1:
        print("Position not yet set")
        fail_cnt += 1
        continue

# check position number
if word2event[word].startswith("Position"):
    pos = int(word2event[word].split("_")[1])
    if position_anchor == pos:
        print("Position not increasing")
        fail_cnt += 1
        continue
    else:
        position_anchor = pos

# check theme region
if myvocab.id2token[word].startswith("Theme"):
    if myvocab.id2token[word] == "Theme_Start" and not previous_labeled:
        previous_labeled = True
        last_theme_bar_idx = bar_count

    elif myvocab.id2token[word] == "Theme_End" and previous_labeled:
        previous_labeled = False
    else:
        print("Theme region error")
        fail_cnt += 1
        continue

# add new event to record sequence

```

```

        words[0].append(word)
        if previous_labeled:
            label_list.append(label_list[-1]+1)
        else:
            label_list.append(0)

        if word2event[word] == "Bar":
            bar_count += 1
            position_anchor = -1
            if bar_count > nBars:
                return words[0]

    fail_cnt = 0

    print ('generated {} events'.format(len(words[0])))
    return words[0]

# load tempo information
with open("./tempo_dict.json") as f:
    tempo_dict = json.load(f)

given_theme = myvocab.midi2REMI(args.theme, theme_annotations=False)
given_theme = [myvocab.token2id["Theme_Start"]] + given_theme + [myvocab.token2id["Theme_End"]]

midiID = os.path.basename(args.theme).split('.')[0].split('_')[0]
print("Tempo from original : {}".format(tempo_dict[midiID]))
tmp = myvocab._tempo_bins[np.argmin(abs(tempo_dict[midiID] - myvocab._tempo_bins))]
given_tempo = myvocab.token2id["Tempo_{}".format(tmp)]

tempo_in_theme = [x for x in given_theme if myvocab.id2token[x].startswith("Tempo")]
if not len(tempo_in_theme) == 0:
    # remove error tempo
    given_theme = [x for x in given_theme if not myvocab.id2token[x].startswith("Tempo")]

model.to(device)
word_seq = inference(
    nBars = args.nBars,
    strategies=['temperature', 'nucleus'],
    params={'t': args.temp, 'p': 0.9},
    theme_seq=given_theme,
    prompt=[given_tempo, myvocab.token2id["Theme_Start"]],
)

# check if no tempo in front , add it
position_events = [i for i, x in enumerate(word_seq) if myvocab.id2token[x].startswith("Position ...")]
if not word_seq[position_events[0]+1] == given_tempo:
    word_seq.insert(position_events[0]+1, given_tempo)

# remove tempo
word_seq = word_seq[1:]

# save to disk
myvocab.REMIID2midi(word_seq, args.out_midi)
print("{} saved".format(args.out_midi))

```

11. Argument Parser — parse_arg.py

Training hyperparameter defaults.

Source Code

```
"""Theme Transformer Arguments

    Author: Ian Shih
    Email: yjshih23@gmail.com
    Date: 2021/11/03

"""
import argparse
parser = argparse.ArgumentParser(description='Theme Transformer Args')

parser.add_argument('--lr', type=float, default=0.0002,
                    help='initial learning rate')
parser.add_argument('--warmup_step', type=int, default=400,
                    help='upper epoch limit')
parser.add_argument('--lr_min', type=float, default=1e-5,
                    help='minimum learning rate during annealing')
parser.add_argument('--clip', type=float, default=3,
                    help='gradient clipping')

parser.add_argument('--max_step', type=int, default=3200000,
                    help='upper epoch limit')
parser.add_argument('--batch_size', type=int, default=8,
                    help='batch size')
parser.add_argument('--max_len', type=int, default=512,
                    help='number of tokens to predict')
parser.add_argument('--seed', type=int, default=1000,
                    help='random seed')
parser.add_argument('--cuda', action='store_true',
                    help='use CUDA')
parser.add_argument('--restart_point', type=str, default='',
                    help='restart_point')

args = parser.parse_args()
```

12. Logger — logger.py

Writes timestamped log lines to both a plain-text file and a colour-coded copy. Helper functions (getRed, getCyan, ...) wrap ANSI escape codes.

Source Code

```
"""Code for logging

    Author: Ian Shih
    Email: yjshih23@gmail.com
    Date: 2021/11/03

"""
import time
import os

def getRed(skk):        return("\033[91m{}\033[00m" .format(skk))
def getGreen(skk):      return("\033[92m{}\033[00m" .format(skk))
```

```

def getYellow(skk):      return("\033[93m{}\033[00m" .format(skk))
def getLightPurple(skk): return("\033[94m{}\033[00m" .format(skk))
def getPurple(skk):      return("\033[95m{}\033[00m" .format(skk))
def getCyan(skk):        return("\033[96m{}\033[00m" .format(skk))
def getLightGray(skk):   return("\033[97m{}\033[00m" .format(skk))
def getBlack(skk):       return("\033[98m{}\033[00m" .format(skk))

class logger():
    def __init__(self, filepath,override=False):
        self.filepath = filepath
        self.colorfilepath = "{}_colored".format(filepath)
        print("Log file at : {}".format(self.filepath))
        if override or not os.path.exists(self.filepath):
            with open(self.filepath, "w") as f:
                f.write("Log created at {}\n".format(
                    time.strftime("%Y/%m/%d %H : %M : %S")))
                print("Log file created : {}".format(self.filepath))
        if override or not os.path.exists(self.colorfilepath):
            with open(self.colorfilepath, "w") as f:
                f.write("Log created at {}\n".format(
                    time.strftime("%Y/%m/%d %H:%M:%S")))
                print("Color Log file created : {}".format(self.colorfilepath))

    def log(self, content, include_header=False, show=True, end='\n'):
        with open(self.filepath, "a") as f:
            if include_header:
                f.write("{} {} ".format(time.strftime("%Y/%m/%d %H:%M:%S")))

            f.write("{}".format(content))
            f.write(end)

        with open(self.colorfilepath, "a") as f:
            if include_header:
                f.write("{} {} ".format(
                    getPurple(time.strftime("%Y/%m/%d %H:%M:%S"))))

            f.write("{}".format(content))
            f.write(end)

        if show:
            print(content, end=end)

if __name__ == "__main__":
    mylogger = logger("./logs/test.log")
    mylogger.log("aasdads")

```

13. Randomness Control — randomness.py

Sets seeds for Python random, NumPy, and PyTorch (CPU + CUDA) to ensure reproducible training and inference runs.

Source Code

```
"""randomness setting code

Fix all random seed

Author: Ian Shih
Email: yjshih23@gmail.com
Date: 2021/11/03

"""

import random
import torch
import numpy as np

def set_global_random_seed(seed):
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    np.random.seed(seed)
    random.seed(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = True
```

14. Architecture Summary

Forward Pass Data Flow

```
Input theme MIDI
  ■
  ▼ vocab.midi2REMI()
theme_token_ids (T,)
  ■
  ▼ WordEmbedding + PositionalEncoding
src (T, 1, d_model)
  ■
  ▼ TransformerEncoder [6 × EncoderLayer]
memory (T, 1, d_model)
  ■
  ■ target token ids (L,)
  ■   ■
  ■   ▼ WordEmbedding
  ■   tgt (L, 1, d_model)
  ■   ■
  ■   ▼ tgt_label (cumulative theme position index)
  ■
  ▼ TransformerDecoder_Hybrid [6 layers]
    Layer 0,1,2 : xor_pattern=0 → standard / CrossOnly
    Layer 3,4,5 : xor_pattern=1 → GPA (parallel + XOR gate)
  ■
  ▼ output_layer Linear(d_model → vocab_size)
logits (L, 1, vocab_size)
  ■
  ▼ temperature + nucleus sampling
next_token_id
```

Decoder Layer Type Comparison

Layer Type	Self-Attn	Cross-Attn	Theme Gate	Use Case
TransformerDecoderLayer	✓	✓ (sequential)	✗	Standard decoder (reference)
TransformerDecoderLayer_CrossOnly	✗	✓	✓ (multiply)	Pure cross-attn with gating
TransformerDecoderLayer_parallel	✓	✓ (parallel)	✓ (XOR gate)	GPA — main innovation

Citation

```
@article{shih2022theme,
  title = {Theme Transformer: Symbolic Music Generation with Theme-Conditioned Transformer},
  author = {Yi-Jen Shih and Shih-Lun Wu and Frank Zalkow and Meinard Müller and Yi-Hsuan Yang} ...,
  journal = {IEEE Transactions on Multimedia},
  year = {2022},
  publisher = {IEEE}
}
```